

Systemnära programmering:

OU3: MDU

Resonemang om trådsäkerhet

Programmet "mdu" kan med hjälp av flaggan "-j" starta och köra programmet med ett godtyckligt antal trådar. Med program som använder sig av trådar så uppstår många potentiella problem. Två vanliga fel är "race condition" som uppstår då flera trådar försöker läsa eller skriva till en delad resurs och "deadlocks" som inträffar då två eller fler trådar väntar oändligt på en resurs, vilket orsakar att programmet ej exekverar.

Inom programmet "mdu" så finns det två olika ställen där ett "race condition" kan uppstå. Det första stället är då ett fil-namn tas bort från kön. Kön är en delad resurs av alla trådar där indata befinner sig och potentiella kataloger som blir upptäckta. Den andra delen av implementationen där ett "race condition" kan uppstå är då storleken på dem specificerade filerna adderas till resultatet.

I implementationen så används synkroniseringsverktyget "mutex". Syftet med "mutex" är att låsa in kritiska delar i en kod för att förhindra "race condition". Ett "mutex" använder sig av två viktiga funktioner. En funktion stänger tillgången för den kritiska delen för alla trådar förutom den tråd som anropade funktionen. Den andra funktionen låser upp den kritiska delen för alla trådar.

Implementationen använder sig av tre olika "mutex" som har två olika syften. Den första "mutexen" används till att förhindra "race condition" till all funktionalitet till kön och den andra används för att förhindra "race condition" vid addition av en fil-storlek till den resulterande storleken. Och den tredje förhindrar "race condition" vid tilldelning av returvärde.

Förutom "race condition" är även "deadlocks" vanliga problem i fler-trådade program. Det finns fyra olika villkor som måste vara uppfyllda för att ett "deadlock" ska inträffa. Dem är namngivna "mutual exclusion", "hold and wait", "no preemptions" och "circular wait".

Det första villkoret "mutual exclusion" innebär att det finns minst en delade resurs som endast en tråd kan använda i taget. Detta har blivit förhindrat med hjälp av "mutex". Varje delad resurs i programmet blir låst av en tråd och låses upp då tråden är färdig med den.

Det andra villkoret "hold and wait" innebär att en tråd som redan håller en delad resurs och har låst tillgången till den och begär åtkomst till en annan delad resurs. Detta har blivit förhindrat genom att skapa en ordning där en tråd endast kan ha tillgång till en delad resurs i taget.

Det tredje villkoret "no preemptions" betyder att en tråd måste frivilligt släppa tillgången till en delad resurs. Detta har förhindrats genom att när en tråd är färdig med en delad resurs frigjort dess "mutex" med samma tråd.

Det fjärde villkoret "circular wait" betyder att en tråd väntar på en delad resurs som en annan tråd använder och den andra tråden väntar i sin tur på en annan delad resurs. Detta medför att alla trådar väntar på något och kan ej fortsätta sitt arbete och gör så att programmet ej exekverar. Detta har inträffar ej då programmet exekverar som det ska.

Eftersom inget av villkoren för att en "deadlock" ska uppstå inträffar så är programmet fri från "deadlocks".

Ytterligare ett fel som kan uppstå när ett program som använder sig av flera trådar och specifikt i implementationen till "mdu" är att om inget fil-namn finns i kön vid tillfället innebär det ej att alla filer har blivit beräknade. Villkoret för att programmet ska exekvera är att kön ska vara tom och antalet aktiva trådar är noll. Detta då en tråd kan hitta underkataloger vid beräkning av kataloger.

För att förhindra att trådar exekverar efter dem utfört en beräkning och se till att trådarna väntar på en fil att beräkna har "mdu" en "condition variabel" implementerats då det är ett effektivt sätt för en tråd att vänta.

Vid användning av "condition variabler" så har "mdu" använt sig av tre av dess funktioner. Den första funktionen är "pthread_cond_wait" som säger till tråden att vänta på en signal ifall kön är tom och det finns minst en tråd som utför en beräkning. Då denna funktion ser till att tråden ej tar upp arbetskraft från processorn vilket är bra.

Den andra funktionen är "pthread_cond_broadcast" som väcker alla trådar som är i väntan på arbete. Denna funktion har använts då ingen fil eller katalog finns att beräkna och antalet aktiva trådar är noll. Detta läge indikerar på att alla beräkningar har blivit utförda och trådarna kan exekvera.

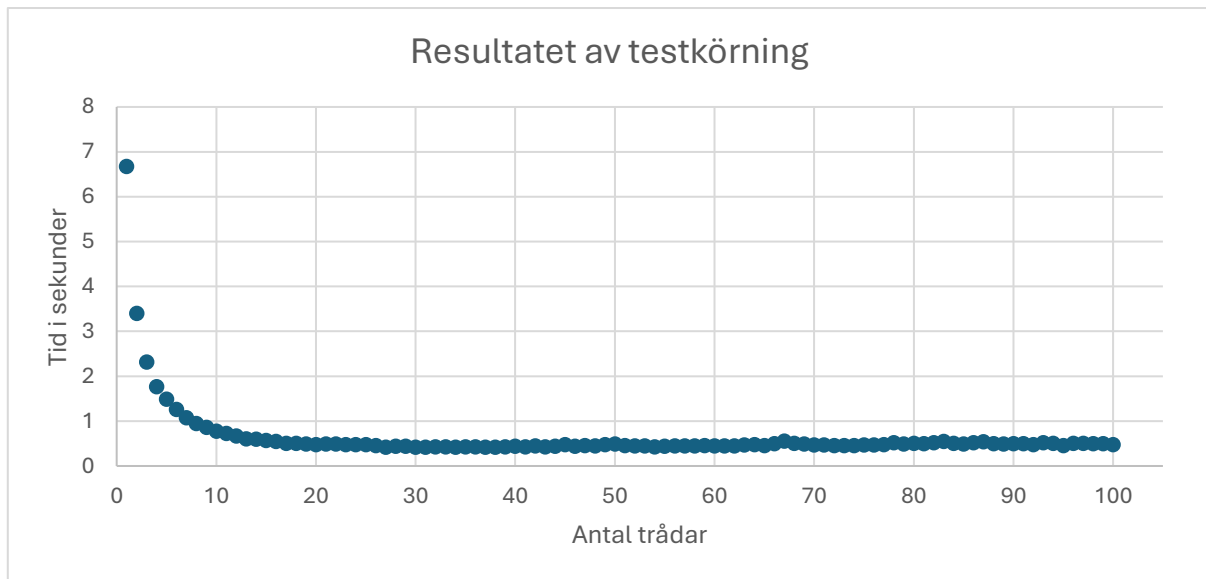
Den tredje funktionen som har använts är "pthread_cond_signal" som signalerar en tråd att det finns en katalog i listan som ska beräknas. Denna funktion har använts i samband då en tråd hittat en underkatalog. Detta är effektivt sätt att signalera då ett "race condition" förhindras om vilken tråd som ska få utföra beräkningen samt ser till att trådar ej exekverar.

Vid testning av trådsäkerhet har verktyget "helgrind" använts som används för analys av trådsäkerhet och identifiering av fel i flertrådade program i programmeringspråket C.

Analys av prestandan

I följande del kommer en analys av prestandan presenteras. En figur innehållande ett resultat som visar hur körtiden då antalet trådar ökar kommer även presenteras. Även ett resonemang om varför körtiden ser ut som den gör kommer även framställas.

Vid testning av körtid har ett script implementerats. Scriptet kör programmet "mdu" n gånger där n är intervallet 1 till 100. Vid varje iteration kommer programmet starta med n trådar. Filen som ska beräknas är katalogen "pkg" lokaliserad på servern "itchy". I figur 1 visas resultatet i form av en graf vid testkörning av scriptet.



Figur 1: Resultatet av testkörning

När programmet använder sig av en tråd så är körtiden 6.68 sekunder vilket är den långsammaste tiden. Den bästa tiden är 0.42 sekunder som framkommer då antalet trådar är 30.

Vid testkörning av "mdu" så blir det en prestandaförbättring då antalet trådar är 1 upp till 30 trådar. Då programmet använder sig av fler än 30 trådar blir tiden långsammare. Detta beror mest troligt på användningen av mutexar.

Som tidigare nämnt så använder trådarna delade resurser vilket medför att implementationen måste använda sig av "mutexar" för att hantera synkronisering. En trolig anledning varför körtiden blir sämre efter 30 trådar kan bero på hur dem kritiska delarna i koden har blivit strukturerade.

Det går inte att garantera att trådarna kan arbeta kontinuerligt. I vissa fall kommer ett antal trådar vänta på en "mutex" ska låsas upp. Om det är flera trådar som väntar på att samma "mutex" ska bli upplåst så tappar alla dem trådarna sin effektivitet. Det verkar som att 30 trådar är ett optimalt antal, eftersom varje tråd då behöver vänta så lite som möjligt på att få tillgång till en mutex, samtidigt som tiden varje tråd spenderar på att vänta på att få bearbeta en katalog minimeras.

En faktor som kan spela roll är skapandet av trådar. Eftersom tidsskillnaden mellan tex 30 trådar och 40 trådar är 0.01 sekunder kan skapandet av trådar vara en faktor då en loop används.

Diskussion och reflektion

Vid implementering av "mdu" har jag stött på mindre problem än tidigare uppgifter. Anledningen till detta tror jag ligger i mer testning av kod och mer faktakontroll om funktioner som skulle användas. Dem största problem jag stött på är "race condition" och korrekt signalering till trådar.

När "race condtion" inträffat har det skett vid beräkning av filer då ett mutex glömt att använts. Jag stötte även på fel signalering till trådar som väntar på en katalog att beräkna. Jag hade använt mig av "pthread_cond_broadcast" i stället för "pthread_cond_signal" när en underkatalog läggs till i kön. Detta gjorde att alla trådar som väntade med hjälp av "pthread_cond_wait" signalerades att en katalog finns och nu ska du utföra en beräkning. Detta resulterade med att en tråd utförde beräkningen och resterande trådar exekverade ur funktionen.

Uppgiften har varit rolig och mycket nytt, då jag aldrig har använt mig av trådar tidigare. Ett spännande verktyg och fascinerande hur effektiva program som kan skapas. Det svåraste med uppgiften är definitivt synkroniseringen så trådarna jobbar korrekt tillsammans. Men ytterligare en gång så vill jag påstå att övningsuppgifterna är för lätta i jämförelse vad som ska implementeras i "mdu".